

MemSOS: OS-Guided Selective Memory Mirroring

Junghoon Kim[†] Jongheon Jeong[†] Seokwon Moon[†]

Seong Hoon Seo[†] Yeonhong Park[†] Jinkyu Jeong[‡] Nam Sung Kim[§] Jae W. Lee[†]

Seoul National University[†], Yonsei University[‡], University of Illinois Urbana-Champaign[§]

[†]{jh0615, jjong0025, moonsw214, andyseo247, ilil96, jaewlee}@snu.ac.kr [‡]jinkyu@yonsei.ac.kr [§]nskim@illinois.edu

Abstract—Memory errors pose an escalating threat to data-center reliability as DRAM technology scales to smaller nodes and servers process ever-larger datasets. While Error Correction Code (ECC) provides a first-line defense, uncorrectable errors still cause catastrophic server failures with significant economic impact. Memory mirroring offers complementary protection against these errors, but existing memory mirroring solutions require reserving specific memory regions exclusively for mirroring, which incurs significant capacity overhead and thus limits wide adoption. A recent proposal suggests leveraging free memory space for mirroring, but it leaves a critical research question unanswered: which data to mirror when free memory is limited. Thus, we propose MemSOS, a selective memory mirroring system that dynamically chooses which pages to mirror based on their impact on system reliability. Specifically, MemSOS selects pages to mirror based on their *criticality* and *recency*. Criticality is evaluated by examining the page type, while recency serves as a proxy for the likelihood of future access. Our evaluation demonstrates that MemSOS reduces system Failures In Time (FIT) by up to 19,000 $\times$ compared to a state-of-the-art partial mirroring scheme, while maintaining less than 3% performance overhead. In many cases, MemSOS achieves reliability levels comparable to full mirroring, underscoring its effectiveness in maximizing system availability under limited free memory space.

I. INTRODUCTION

Memory reliability is a critical challenge for datacenters. As the volume of data managed by datacenters continues to grow and technology scales down, the frequency of memory errors has increased [54], [63], [65], [70]. Unhandled memory errors can result in server failures, potentially leading to costs in the millions of dollars [4], [38], [45], [51]. This issue is especially significant for services managing sensitive data, such as financial, medical, and customer information, where loss or corruption of data could cause severe socio-economic ramifications. Enhancing the memory reliability of systems is an essential mission for datacenters.

Although Error Correction Code (ECC) [9], [13], [34]–[36], [39], [55], [59], [72] is the standard technique for memory reliability, mirroring offers complementary protection that works synergistically with ECC. By storing redundant copies, it recovers from errors even uncorrectable by advanced ECC schemes like Chipkill [13], such as double-symbol errors and memory channel or controller failures. Modern servers already support mirroring in hardware [21], [29], with vendor tools available for configuration [12], [17], [43]. However, memory mirroring comes with the significant drawback of memory capacity overhead, which limits its widespread adoption. Such implementations dedicate fixed memory regions to mirrors,

TABLE I: Design Comparison of Memory Mirroring Schemes

	Flexible Mirror Space	Criticality-Aware	Recency-Aware
Lenovo [44]		$\triangle$	
Dvé [61]	✓		
MemSOS	✓	✓	✓

reducing usable capacity by up to 50% in full mirroring modes [21], [23], [25], [29], [44].

To address this limitation, Dvé [61] recently proposed leveraging free memory for mirroring. Nonetheless, this solution primarily focuses on scenarios with sufficient free memory for full mirroring, leaving a critical question unanswered: What should be mirrored when free memory is limited? In practice, memory utilization can vary significantly depending on workload characteristics, sometimes reaching near full capacity. Developing an effective mechanism to prioritize which data to mirror is essential for maintaining system reliability, even when the available memory varies.

We address this challenge with MemSOS, a selective memory mirroring solution designed to ensure high reliability even when free memory is constrained. MemSOS implements a page-granularity policy that selects data to mirror based on two key metrics: criticality and recency. It evaluates criticality by analyzing page type (*e.g.*, kernel, file-backed, or anonymous) and leverages the Least Recently Used (LRU) mechanism to estimate recency, prioritizing pages likely to be accessed in the near future. Since memory faults only cause errors upon access, this prioritization reduces error observability and enhances system resilience. MemSOS extends the OS with a lightweight daemon to manage mirror selection and requires only minimal modifications to the memory controller to support mirror creation, removal, and redirection. As shown in Table I, MemSOS is the first system to combine flexible mirror space with selective mirroring based on both criticality and recency, providing significantly greater flexibility than existing solutions.

We evaluate MemSOS on representative real-world workloads and find that under severe memory pressure (*i.e.*, 10% free memory), MemSOS improves system reliability—as measured in Failures In Time (FIT)—by up to 19,000 $\times$ over Lenovo’s partial mirroring [44]. In many cases, MemSOS achieves reliability comparable to full mirroring with negligible performance overhead (<3%).

Our key contributions include:

- MemSOS provides a comprehensive policy that determines which data to mirror under limited free memory by leveraging operating system knowledge to evaluate both the criticality and the recency of a memory page for quantifying its impact on system reliability.
- MemSOS introduces modular and modest modifications to the operating system and memory controller, making it a practical solution that can readily be deployed to real systems. We can observe reliability improvements with negligible performance degradation of 3% or less.
- We demonstrate the effectiveness of MemSOS by evaluating it on representative real-world datasets, achieving up to 19,000 $\times$ improvement in terms of FIT compared to existing partial mirroring approaches.

II. BACKGROUND AND MOTIVATION

A. Memory Errors

Terminology. In this paper, we adopt the terminology commonly used in previous studies [19], [33], [34] on DRAM reliability. We specifically distinguish between three key concepts: fault, error, and failure. A *fault* refers to an unintended disruption in hardware behavior (*e.g.*, hardware defects). For example, if a specific I/O pin experiences a malfunction that causes its value to be permanently stuck at 1, it qualifies as a fault. A fault can lead to an *error*, which occurs when the retrieved data differ from the original data. However, not all faults necessarily result in errors—for instance, if the original value was 0, a 0-stuck fault would not cause a discrepancy, meaning no error would be observed. When an error occurs, it is considered a *failure* if it propagates to the system level and becomes observable by the user.

Memory Error Handling. Memory systems employ hardware mechanisms to detect—and in some cases, correct—errors. DRAM modules commonly use ECC, which can detect and correct single-bit and multi-bit errors depending on the implementation. Additionally, Cyclic Redundancy Check (CRC) protects memory channels against transmission errors. Similar to faults in other hardware components (*e.g.*, CPUs), the system’s response to memory faults depends on their detectability and correctability. Figure 1 illustrates the four possible scenarios. First, when errors remain undetected—known as silent data corruption (SDC)—the system cannot respond and continues operating with corrupted data (❶). These errors prove particularly dangerous as corruption propagates through the system unnoticed. Second, when hardware mechanisms like ECC detect and correct errors, the system resumes normal operation without disruption (❷). Third, for uncorrectable but recoverable errors, the system can restore data from alternative sources—for instance, re-reading clean file-backed pages from disk—enabling continued operation (❸). Finally, uncorrectable and unrecoverable errors lacking alternative data sources trigger system-level failures, ranging from process termination to system crashes (❹).

Impact of Memory Errors in Datacenters. As memory cells shrink with technology scaling, they suffer increasingly from

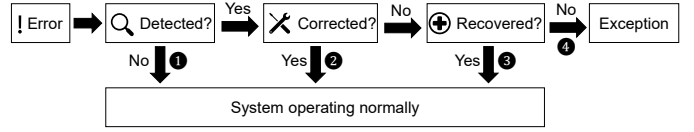


Fig. 1: Memory Error Handling

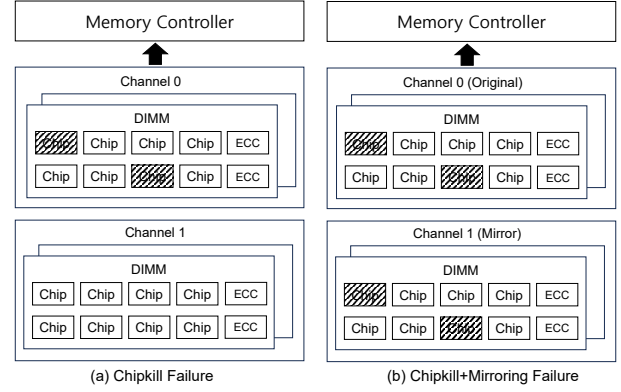


Fig. 2: Failure Examples

reduced noise margins, elevated cell-to-cell interference, and higher leakage currents [54], [63], [65], [70]. These reliability challenges have become a critical concern for servers. A 2024 industry survey [31] reported that 90% of mid-sized and large enterprises experience server downtime costs exceeding \$300,000 per hour, with 41% reporting potential losses of millions of dollars per hour. While downtime stems from various causes, extensive field studies [4], [38], [45], [51], [53], [73] have identified memory errors as a leading contributor to server failures, particularly in datacenter environments.

B. Error Correction Code (ECC)

To address these failures, modern systems employ ECC mechanisms [9], [13], [34]–[36], [39], [55], [59], [72], which detect and correct errors using redundant information in the form of check bits. Conventional ECC schemes, such as Single Error Correction-Double Error Detection (SEC-DED), offer basic protection but may not adequately address multi-bit faults and chip-level failures. As memory density increases, stronger error correction techniques are essential to ensure system reliability and availability in large-scale deployments.

Rank-Level ECC. Rank-Level ECC enhances memory reliability by storing check bits in additional memory chips within a rank. Depending on the implementation, it can range from SEC-DED, which corrects single-bit errors and detects double-bit errors, to more advanced schemes like Chipkill [13]. Chipkill employs Single Symbol Correcting-Double Symbol Detecting (SSC-DSD) codes, allowing the system to tolerate the failure of an entire chip within a rank. By providing stronger protection against multi-bit errors and chip failures, Rank-Level ECC ensures greater system stability and reliability in large-scale computing environments.

TABLE II: Failures in Time (FIT) for Chipkill and Full Mirroring

	FIT (per billion device-hours)
Chipkill	$45 \times FIT_{\text{chip}}^2 \times 10^{-9} \times r$
All (Chipkill+Full Mirroring)	$1260 \times FIT_{\text{chip}}^4 \times 10^{-27} \times r$

C. Memory Mirroring

Mechanism. Memory mirroring is a technique that enhances memory system reliability by maintaining redundant copies of data, allowing recovery in the event of an uncorrectable error. In essence, mirroring reduces system failures caused by errors by converting cases from the fourth scenario (4) to the third scenario (3) in Figure 1. Typically, memory mirroring places mirrored data on a different memory channel from the original data to provide maximum fault isolation.

Server CPU vendors have incorporated hardware mechanisms to support mirroring in their products. For example, Address Range Memory Mirroring [21] in Intel Xeon Scalable Processors [29] allows a portion of the memory to be reserved for mirroring through BIOS configurations. Leveraging such mechanisms, server manufacturers [12], [17], [43] offer software tools to enable memory mirroring functionality in their servers.

Synergy with ECC. Mirroring complements ECC by recovering from uncorrectable errors that ECC alone cannot handle, significantly enhancing overall reliability. While ECC corrects errors within individual chips, mirroring recovers from any uncorrectable errors provided the errors do not simultaneously affect both the original and mirrored data, resulting in high system reliability through synergy with ECC. Figure 2(a) illustrates a scenario where two chips in the same DRAM module fail simultaneously. In this case, Chipkill alone results in data loss, whereas mirroring successfully recovers the data using the mirrored data. Figure 2(b) further emphasizes that mirroring fails only if simultaneous errors affect both the original and mirrored data, highlighting the complementary reliability benefits provided by mirroring. Additionally, mirroring can address errors detected in other parts of the memory subsystem, such as the memory channel or memory controller, which ECC alone cannot correct [51], [61], [64].

Following the methodology from previous work [61], we quantify the reliability benefits of mirroring in terms of FIT, defined as failures per billion device-hours. Our analysis assumes full mirroring, where all data are mirrored, with DDR5 DIMMs with 10 chips per rank (eight for data and two for Chipkill). Table II presents the results, where FIT_{chip} denotes the FIT of a single chip and r represents the number of ranks in the system. Mirroring significantly reduces FIT, achieving reductions by several orders of magnitude. Specifically, mirroring reduces FIT by a factor of $(0.0357 \times 10^{18})/FIT_{\text{chip}}^2$. The equations below describe in detail how the FIT of each scheme is derived.

For Chipkill-only protection, a failure occurs when any 2

out of 10 chips in a rank fail:

$$\frac{FIT_{\text{chipkill}}}{10^9} = \frac{10C_2 \times FIT_{\text{chip}}^2}{10^9 \times 10^9} \times r$$

For Chipkill + Full Mirroring, we consider the case where two chips fail in the original channel and two chips fail in its mirrored channel. A failure occurs if there is any overlapping chip index between the failed chips in the original and the mirrored channel—that is, if at least one chip with the same index fails in both:

$$\frac{FIT_{\text{all}}}{10^9} = \frac{(10C_2 \times 10C_2 - 10C_2 \times 8C_2) \times FIT_{\text{chip}}^4}{10^9 \times 10^9 \times 10^9 \times 10^9} \times r$$

Note that this quantitative analysis considers only errors occurring in DRAM. The effective improvement in memory reliability could be even higher if errors in other parts of the memory subsystem, beyond DRAM—such as memory channels or controllers—are taken into account, as these are errors that mirroring can address but ECC cannot.

Challenges in Memory Mirroring. Despite its reliability benefits, traditional memory mirroring incurs significant memory overhead, limiting its widespread adoption. Prior work, such as Dvé [61], addresses this by dynamically replicating data to underutilized memory, enabling on-demand mirroring based on available capacity. While Dvé provides an effective solution for leveraging idle memory, it mirrors all pages uniformly without considering their criticality or access characteristics. However, available free memory in production datacenters is typically limited and fluctuates [32], [40], [49], [62], necessitating selective mirroring mechanisms that adapt to memory pressure—a critical capability that Dvé does not provide.

To address this, industry solutions including Lenovo’s [44] adopt a selective mirroring strategy that uses a reserved mirror region, prioritizing the OS kernel—which is particularly critical in the event of a failure—over user-space data. If additional memory remains after mirroring the kernel, user-space pages are also mirrored, as space allows. However, this approach does not differentiate among user-space regions, overlooking that they may have varying characteristics relevant to mirroring under limited memory. This highlights the need for a more fine-grained selective mirroring policy.

III. MEMSOS OVERVIEW

Addressing the limitations of traditional memory mirroring, we propose MemSOS, a novel OS-guided selective memory mirroring scheme designed to significantly reduce spatial overhead and enhance reliability under extreme memory constraints. MemSOS introduces two key innovations: (1) a fine-grained, page-level mirroring mechanism that dynamically leverages available memory space, and (2) an OS-guided policy that intelligently selects pages to mirror based on their criticality and recency. These innovations enable MemSOS to provide high memory reliability even in environments with severely limited free memory.

Figure 3 provides an overview of MemSOS, highlighting its four components shaded in gray: Mirror Selection Daemon,

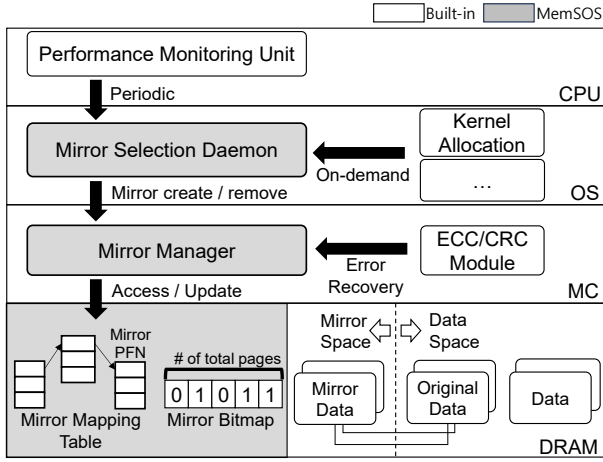


Fig. 3: MemSOS Overview

Mirror Manager, Mirror Mapping Table and Mirror Bitmap. In the following, we describe the organization and operation of each component.

Mirror Selection Daemon. MemSOS introduces a new OS-level component, Mirror Selection Daemon, which determines which data to mirror based on the amount of available free memory. Mirror Selection Daemon updates mirrored regions in two ways: (1) periodically, based on memory access history collected via hardware features such as the Performance Monitoring Unit (PMU)—including Intel’s Processor Event-Based Sampling (PEBS) [28] and AMD’s Instruction-Based Sampling (IBS) [14]—and (2) immediately, in response to critical events such as kernel memory allocations. In both cases, Mirror Selection Daemon identifies which pages to mirror or unmirror and sends the corresponding requests to Mirror Manager. Section IV provides further details on Mirror Selection Daemon.

Mirror Manager. Mirror Manager is an augmentation to the memory controller that performs three main tasks. First, it handles the creation and removal of mirror space in response to requests from Mirror Selection Daemon. Second, it manages error recovery by retrieving mirrored pages. Third, it processes write requests. In a system with mirroring, whenever a write request is received, the system must check whether the destination page is mirrored. If the page is mirrored, the write operation must be performed on both the original and mirrored pages. Communication between the OS and Mirror Manager uses standard memory-mapped I/O (MMIO), through which the OS conveys the relevant physical frame number (PFN) information for mirror creation and removal requests. This approach requires no ISA modifications and follows the conventional OS–hardware coordination model used in prior work [60], [74]. Section V describes the details of Mirror Manager.

Mirror Mapping Table and Mirror Bitmap. Mirror Mapping Table and Mirror Bitmap are memory-resident meta-data structures introduced by MemSOS to efficiently manage mirrored pages. Mirror Mapping Table is implemented as

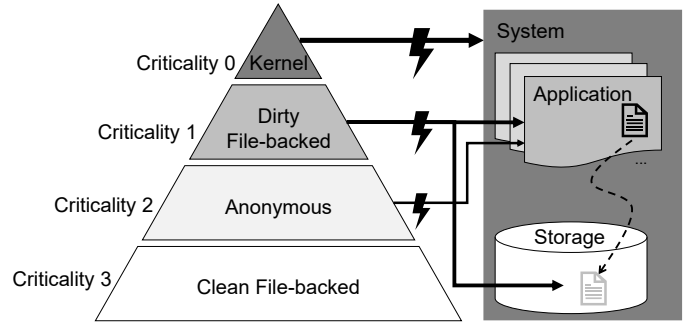


Fig. 4: MemSOS Criticality of Different Pages

a multi-level table that dynamically maps original physical addresses to their mirrored counterparts, significantly reducing memory overhead via sparse allocation. Alongside Mirror Mapping Table, Mirror Bitmap is stored as a compact bitmap that enables quick lookups to determine whether a page is mirrored, using one bit per physical page. In the worst-case scenario, where up to half of the system memory is mirrored, a 1TB memory system with 4KB pages and 52-bit physical addresses would involve managing approximately 128 million mirrored pages. In this case, Mirror Mapping Table would require approximately 640MB (128 million pages $\times$ 5 bytes per entry), representing about 0.064% of the total memory. Mirror Bitmap would additionally consume roughly 32MB (256 million pages $\times$ 1 bit), accounting for around 0.003%. In total, both memory-resident structures combined occupy less than 0.07% of the overall system memory, ensuring minimal spatial overhead.

IV. MIRROR SELECTION DAEMON IN OS

A. Mirror Selection Policy

To prioritize pages for selective mirroring, Mirror Selection Daemon evaluates each page based on its *criticality* and *recency*, following a two-level priority scheme. The criticality of each page reflects the scope and severity of its impact on the system in the event of a failure, making it natural to prioritize pages with higher criticality. Among pages with the same criticality, Mirror Selection Daemon prioritizes those expected to be accessed soon. Ideally, mirroring should follow the order of future reuse distance, but since accurately determining reuse distance is infeasible, Mirror Selection Daemon uses recency as a proxy. In summary, Mirror Selection Daemon first ranks pages by criticality, mirroring those with the highest criticality first. When mirroring space is insufficient to cover all pages within a criticality level, Mirror Selection Daemon then uses recency to prioritize among them.

Mirror Selection Daemon assigns one of four criticality levels to each page, as shown in Figure 4. Criticality 0 is given to kernel pages that can cause a system-wide kernel panic. Criticality 1 is assigned to dirty file-backed pages, where errors can lead to process termination and potential data loss. Criticality 2 includes anonymous pages, which may also cause process termination but without permanent data loss.

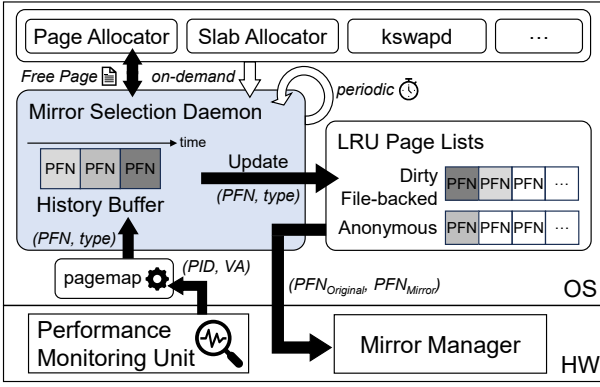


Fig. 5: Mirror Selection Daemon

Criticality 3 consists of clean file-backed pages, which can be recovered from disk without mirroring. Therefore, Criticality 3 pages are excluded from mirroring even if space is available.

To evaluate recency, Mirror Selection Daemon employs the LRU strategy. By selecting recently accessed pages that are likely to be reused soon, MemSOS mirrors pages where latent faults are expected to be exposed sooner, thereby reducing the risk of unexpected failures and prolonging system operation.

B. Operations of Mirror Selection Daemon

Figure 5 provides an overview of the operations of the Mirror Selection Daemon. Mirror Selection Daemon collects and manages data to evaluate the recency of pages. Upon activation, it selects pages for mirroring and sends requests to Mirror Manager accordingly. The rest of this section details the metadata managed by Mirror Selection Daemon, its memory access profiling methods, and its approach to selecting pages for mirroring and updating mirrors.

Mirror Selection Daemon Metadata. Mirror Selection Daemon maintains a history buffer, a one-dimensional array that records recent memory accesses, where each entry contains PFN of the accessed page. It uses this buffer to update the kernel’s LRU lists and assess the recency of *mirrorable pages*—defined as all pages except clean file-backed pages. For file-backed pages, the kernel moves dirty pages to the head of the LRU list, enabling accurate recency tracking for this page type. In contrast, kernel pages, which have the highest criticality, are always mirrored and thus excluded from LRU-based prioritization. Pages pinned with `mlock` reside outside the normal LRU lists and would otherwise be excluded from mirroring. MemSOS addresses this by forcibly placing each pinned page at the head of its file or anonymous LRU list, ensuring it is always considered.

Memory Access Pattern Profiling. Because the kernel’s pseudo-LRU policy updates page recency only when `mark_page_accessed()` is explicitly invoked—such as on page faults, file cache hits, or reclaim scans—it may miss some recent accesses. Refining recency by periodically traversing accessed bits would incur costly page table walks and reverse mapping traversals over many LRU-managed pages. Instead, MemSOS uses PMU sampling to monitor a subset of

memory accesses (e.g., LLC-load-misses) and update recency for sampled pages, thereby enabling low-overhead tracking.

Specifically, Mirror Selection Daemon leverages the PMU [14], [28] to log one address every R LLC misses, translating sampled virtual addresses and PIDs to PFNs via pagemap. The sampling period R balances profiling accuracy and overhead: smaller values improve fidelity but increase interrupt frequency. By default, MemSOS sets $R=10,000$ to keep CPU overhead below 1%, and adapts it between 1,000 and 50,000 based on how quickly the history buffer fills during each 200ms update. If the buffer nears its 8 MB limit (as it might at $R=1,000$ under a 300 GB/s bandwidth), Mirror Selection Daemon increases R to reduce load; if growth is slow, it decreases R to improve precision. We evaluate this adaptive policy in Section VII-C.

Periodic Mirror Update. Mirror Selection Daemon selects pages for mirroring based on their criticality and recency. It is activated at fixed time intervals, and it updates the LRU lists using the profiled memory accesses stored in the history buffer. Subsequently, Mirror Selection Daemon selects pages according to the two-level priority scheme, requests free pages from the page allocator and issues mirror creation or removal requests accordingly. To control the overhead of mirror creation, the creation rate is limited to 2000 mirrors per second, and its impact is evaluated through a sensitivity study in Section VII-C.

On-demand Mirror Update. Mirror Selection Daemon is triggered by events requiring immediate mirror updates, such as memory allocations, mirrorable page removals, or page-reclaim requests. It updates mirrors in-place without clearing the history buffer. When a new page is allocated—either for kernel or user memory—Mirror Selection Daemon may evict low-priority user mirrors to make room. If insufficient, it reclaims user memory via swap or flush. However, Mirror Selection Daemon never unmirrors kernel pages, preserving protection for critical kernel data even under memory pressure. When a mirrorable page is freed (e.g., by deallocation), its mirror is also removed, with replacement deferred to the next periodic update to limit overhead.

C. Example Walk-Through

Figure 6 presents an example walk-through of the operations of Mirror Selection Daemon. The process begins at time T_0 , immediately after selection, with the history buffer cleared. At this point, six pages are selected for mirroring. Subsequently, between T_0 and $T_0 + T$, five pages are sampled in the history buffer. At $T_0 + T$, Mirror Selection Daemon updates the LRU lists using the history buffer, and as a result of mirror selection, a single mirror is replaced; the mirror for Page 23 is removed and Page 31 is newly mirrored. A similar process occurs between $T_0 + T$ and $T_0 + 2T$, but the decreased memory utilization enables up to 9 pages to be mirrored, resulting in 4 more mirrors for Pages 21, 17, 23, and 8 compared to T_1 . At T_1 , T_2 , and T_3 , Mirror Selection Daemon is activated on-demand, each due to the flush of a dirty file-backed page, the kernel page allocation, and the page reclaim request. In these

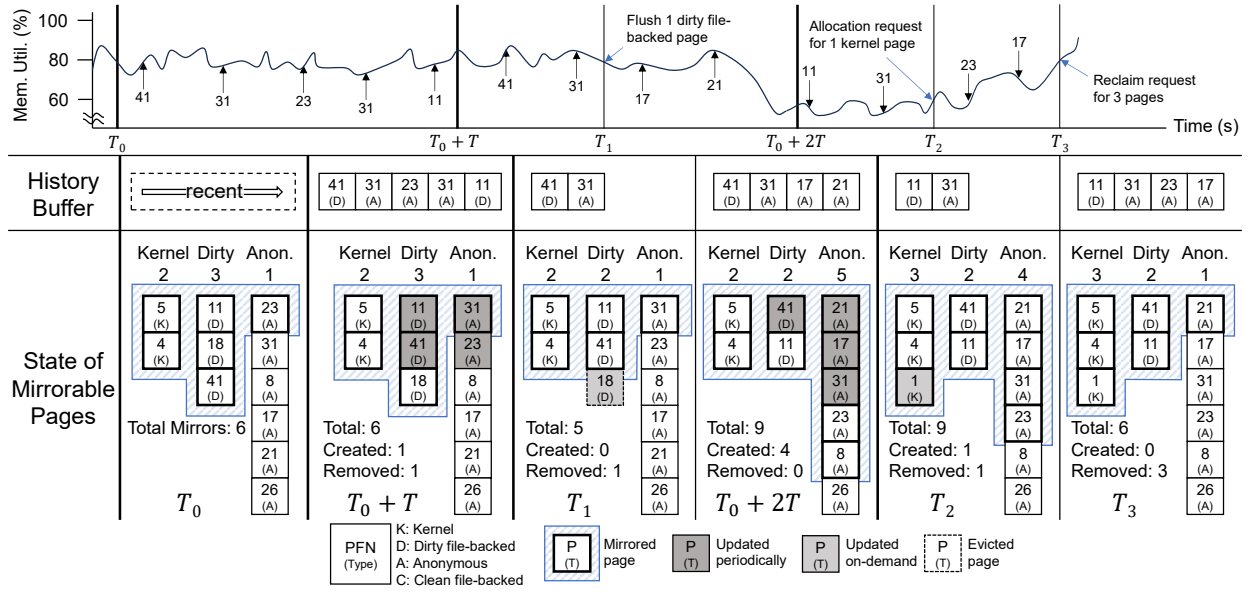


Fig. 6: Example Walk-Through of Mirror Selection Daemon Operations

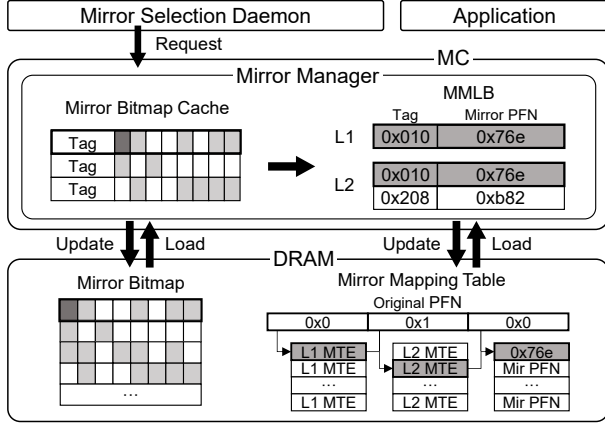


Fig. 7: Mirror Manager

cases, the history buffer remains unchanged, and the mirror creation and removal requests are directly processed. At T_1 , the mirror for the flushed file-backed page 18 is removed, and at T_2 , the mirror is replaced, from page 8 to the newly allocated kernel page 1. At T_3 , soaring memory utilization forces Mirror Selection Daemon to free memory used for mirroring. Consequently, the mirrored pages with the lowest priority (Pages 17, 31, and 23) lose their mirrors.

V. MIRROR MANAGER IN MEMORY CONTROLLER

Mirror Manager is a mechanism within the memory controller responsible for managing the creation and removal of mirrored pages, as well as handling error recovery and write requests. For handling error recovery and write requests, two operations are necessary: first, checking if a specific page is being mirrored, and second, determining the address of the mirrored page if the page is mirrored. These tasks can be performed by consulting Mirror Bitmap and Mirror Mapping

Table, respectively, but frequent access to DRAM-resident structures could cause performance degradation.

To minimize DRAM accesses to the Mirror Bitmap and Mirror Mapping Table, Mirror Manager maintains caches for each, as shown in Figure 7. Mirror Bitmap Cache contains 16K entries, each holding 8 bits of data along with a tag. This gives Mirror Bitmap Cache a total size of 60KB. The Mirror Mapping Lookaside Buffer (MMLB) is analogous to a translation lookaside buffer (TLB), caching recently accessed mapping pairs from the Mirror Mapping Table. It is organized into two levels for improved efficiency. MMLB and Mirror Bitmap Cache are managed separately because they serve different access patterns. MMLB is accessed only when a mirror exists, whereas Mirror Bitmap Cache can be accessed regardless of whether a mirror is present. This separation ensures efficient handling of mirror-related metadata while optimizing overall performance. In our prototype, Mirror Manager logic is integrated within the Caching and Home Agent (CHA) [26], [30], which interfaces directly with multiple DRAM channels through the memory controller. For simplicity, we refer to this unit as the memory controller throughout the paper. This placement allows MMLB and Mirror Bitmap Cache to operate at the system-controller level across interleaved channels without requiring inter-controller communication. The following paragraphs explain how Mirror Manager performs its operations.

Mirror Creation and Removal. Figure 8(a) illustrates the mirrored page creation process. Creation occurs sequentially, one page at a time. First, the original data is copied by reading from the original source (1) and writing to the mirror location (2), processing all 64 cache lines within a page in ascending order. Next, the page access hits at MMLB, and the corresponding entry is updated; in the case of an MMLB miss, a new entry is allocated and updated (3). The mirror status

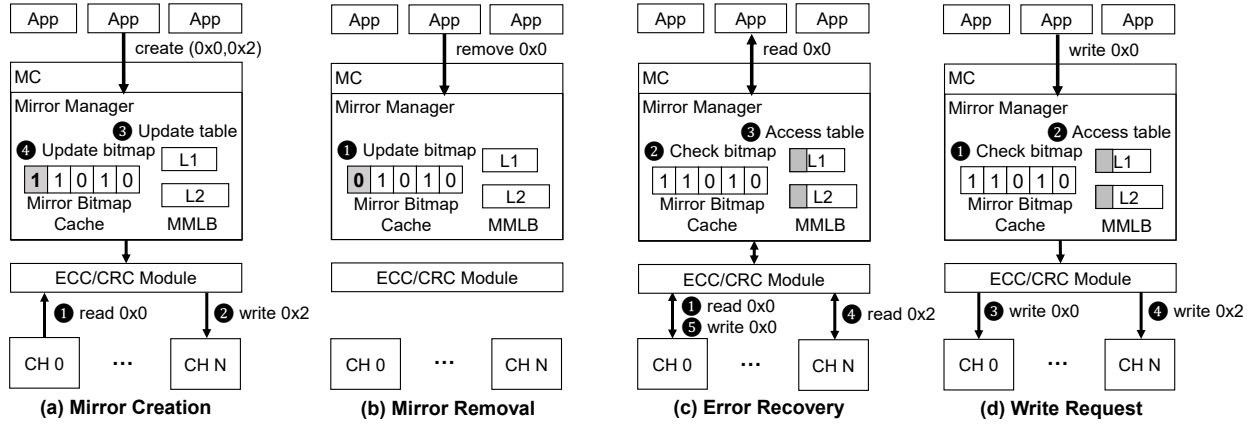


Fig. 8: Illustration of Mirror Manager Handling in MemSOS

is then set: on cache hit, the Mirror Bitmap Cache entry is set to 1; on cache miss, the Mirror Bitmap is first loaded and then set to 1 (④). Figure 8(b) shows mirrored page removal, which simply requires resetting the Mirror Bitmap Cache entry to 0 (①). The corresponding MMLB entry does not need to be invalidated, as it is accessed only when the corresponding Mirror Bitmap Cache entry is 1.

Error Recovery. Figure 8(c) depicts the error recovery process executed by Mirror Manager. When ECC or CRC module detects an uncorrectable error during a read request (①), Mirror Manager first checks Mirror Bitmap Cache. If there is a cache hit, it proceeds; otherwise, Mirror Bitmap is consulted to verify if a mirror exists for the page containing the requested data (②). If a mirror is found, Mirror Manager retrieves the address of the corresponding mirrored page (③). This is achieved by checking the MMLB for a hit; if not, Mirror Mapping Table is accessed. After determining the mirror address, Mirror Manager issues a mirror read request to retrieve the data from the mirrored page (④). Finally, the original data is restored by writing back the retrieved data from the mirror to its original location (⑤).

Handling Write Requests. Figure 8(d) illustrates how Mirror Manager processes DRAM write requests. First, Mirror Manager checks Mirror Bitmap Cache (and Mirror Bitmap on a miss) for the page containing the requested data to determine if a mirror exists (①). If a mirror is present, both the original page and its mirror must be updated to maintain consistency. To accomplish this, Mirror Manager checks MMLB (and Mirror Mapping Table on a miss) for the mirrored page address (②). Once the mirror address is obtained, Mirror Manager issues two DRAM write requests: one for the original page and one for the mirrored page (③ and ④).

In rare cases, concurrent write requests during page mirroring can lead to inconsistencies between the original and mirrored data if not properly managed. To ensure consistency, the DRAM arbitrator defers writes to cache lines currently being copied, resuming them only after the copy completes. It uses an 8-byte SRAM-based flag—one bit per cache line—to track copy status across all 64 lines in a page. The flag is

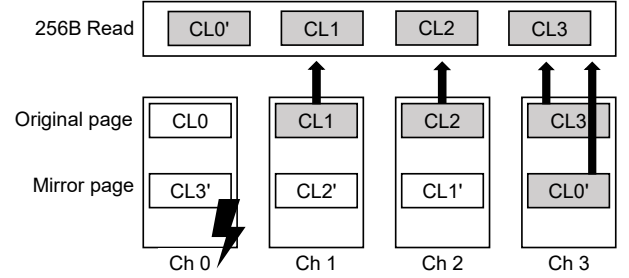


Fig. 9: Shuffling for non-DRAM Error

reset at the start of each mirror creation and updated as lines are copied. This lightweight mechanism guarantees correctness with minimal overhead.

Channel Bit Shuffling for Channel-Level Fault Tolerance. Figure 9 illustrates how mirrored pages are distributed in a 4-channel system. For each cache line, the 2-bit channel index is inverted (bitwise NOT), ensuring that original and mirrored lines are placed on different channels within the same rank. This hardware-managed interleaving balances load across channels and enables fault isolation without OS involvement. This design guarantees channel-level fault tolerance: when a channel fails (e.g., Channel 0), the corresponding mirror (e.g., in Channel 3) preserves data availability. This mechanism protects against both DRAM and non-DRAM faults, such as interface failures, by ensuring original and mirror lines are physically separated. While Mirror Selection Daemon selects which pages to mirror, placement is handled transparently by Mirror Manager in the memory controller, requiring no OS awareness of physical layout.

VI. METHODOLOGY

A. System Setup and Workloads

System Setup. We implement MemSOS in the Linux kernel v5.15.0 and evaluate it on a dual-socket server, with each socket equipped with an Intel Xeon Gold 6426Y CPU (16 cores) and eight 64GB DDR5 DIMMs. Although the system supports dual-socket NUMA, we use a single socket

TABLE III: System and MemSOS Configurations

System Configurations		
CPU	Name	Intel Xeon Gold 6426Y
	# of Cores	16
	L3 Cache	37.5MB
	Max Freq.	4.10GHz
DRAM	Type	DDR5 (4800MT/s)
	Total Size	512GB (8×64GB DIMM)
	Peak BW	307.2GB/s
OS		Linux kernel v5.15.0
MemSOS Configurations		
Mirror Mapping	L1	0.5625KB (64 entries)
Lookaside Buffer	L2	8KB (1024 entries)
Mirror Bitmap Cache		60KB (16384 entries)

TABLE IV: Workload Characteristics

	File page ratio	Anon. page read ratio	Kernel page read ratio	Total write rate
DeathStarBench				
Hotel Reservation (HR)	Low	Low	Medium	Medium
Social Network (SN)	Low	Low	Medium	High
CloudSuite 4.0				
Data Serving (DS)	High	High	Low	Medium
Web Serving (WSv)	Medium	Low	High	Low
Web Search (WSr)	Low	High	Low	Medium

to eliminate cross-node interference and isolate MemSOS’s effects. MemSOS needs no modification for NUMA: each PFN belongs to one node, so our page-based mirroring naturally inherits NUMA awareness from the OS memory allocator and avoids cross-node traffic.

To assess the impact of mirroring on reliability and performance, we test with 307.2GB, 384GB, and 460.8GB of allocated memory, corresponding to 60%, 75%, and 90% of total system memory utilization, respectively. Memory utilization encompasses both kernel and user-space memory. While the kernel memory footprint varies with the workload and system utilization, it typically ranges from 0.5% to 3.0% of the total system memory. The remaining memory is allocated to user applications and page caches, comprising their working set.

TABLE V: Fault model configurations. Component fault rates are given in FIT/device.

Component Fault Rates		
mode	transient	permanent
single bit & single word	25.14	39.34
single column	5.26	2.91
multiple column	7.2	0.92
single row	1.06	8.06
multiple row	2.27	2.24
single bank	0.16	0.15
multiple bank	0.09	0.18

Inherent Faults Parameters		
	intermittent	permanent
rate	10^{-5}	10^{-9}
activation prob.	10^{-6}	1

Workloads. We evaluate five workloads from DeathStarBench [18] and CloudSuite 4.0 [58], representing both traditional cloud services and emerging microservice architectures. These workloads exhibit diverse memory access characteristics—including file page ratios, anonymous and kernel page read patterns, and overall write rates—which directly affect

system reliability and performance. Table IV summarizes their key properties.

B. Comparative Baselines

We compare MemSOS against three baselines: *Chipkill-only*, *Lenovo*, and *Full Mirroring*. All configurations assume Chipkill is enabled. *Chipkill-only* performs no mirroring. *Full Mirroring* duplicates all memory for maximum reliability at a $2\times$ capacity cost. *Lenovo* mirrors memory at coarse address ranges, prioritizing the hypervisor and system processes while treating user space uniformly. In our container-based setup (CloudSuite and DeathStarBench), we emulate this by mapping the hypervisor domain to kernel space and randomly selecting user pages. For fairness, we integrate Lenovo’s prioritization with Dvé’s [61] flexible memory use, creating a hybrid baseline that preserves Lenovo’s hierarchy while enabling adaptive mirror capacity.

We evaluate two variants of MemSOS: *Criticality-only* and *Full*. Both prioritize critical pages foremost. The Criticality-only version selects pages solely by criticality. The Full version adds recency-aware selection, prioritizing recently accessed pages within each criticality level, ensuring frequently accessed critical pages are mirrored under memory pressure.

C. Reliability Simulation Methodology

To evaluate reliability, we use DRAM FaultSim [33], which estimates FIT by modeling two types of DRAM faults: *component faults*, caused by operational defects (e.g., sub-wordline drivers, sense amplifiers), and *inherent faults*, such as those due to Variable Retention Time (VRT), which are present from production and persist throughout the device’s lifetime. 1Component faults are categorized as transient (soft) or permanent (hard), while inherent faults include intermittent and permanent types. These faults are modeled by their ratios and activation probabilities using DDR5 parameters from DRAM FaultSim [33]; detailed values are listed in Table V. The modeled FIT trends are in line with field-study observations [2].

Meanwhile, the vanilla DRAM FaultSim derives FIT assuming generic, randomly-generated memory access patterns. To evaluate reliability under realistic workloads listed in Section VI-A, we extended the frontend of DRAM FaultSim to accept memory access traces as input. Additionally, we assume that the OS is equipped with a recovery mechanism capable of handling errors in clean file-backed pages for both MemSOS and the comparison baselines. Consequently, when quantifying failures, we exclude errors occurring in clean file-backed pages.

D. Performance Evaluation Methodology

We divide evaluation into two categories: hardware-centric and software-centric operations. For hardware-centric operations (mirror creation and mirrored-write handling), direct measurement is infeasible due to limited visibility into the memory controller. Instead, we collect memory access traces using PMU sampling and replay them with injected DRAM

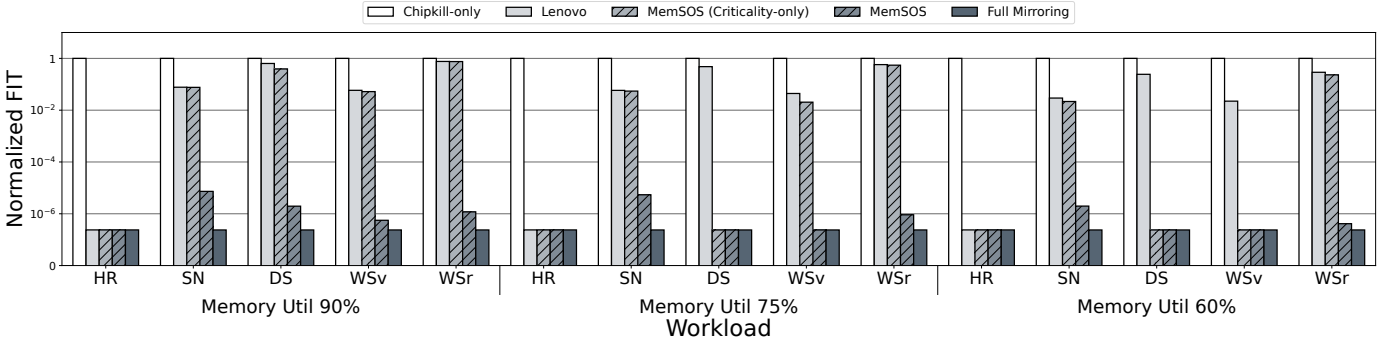


Fig. 10: FIT normalized to Chipkill-only baseline. Lower is Better.

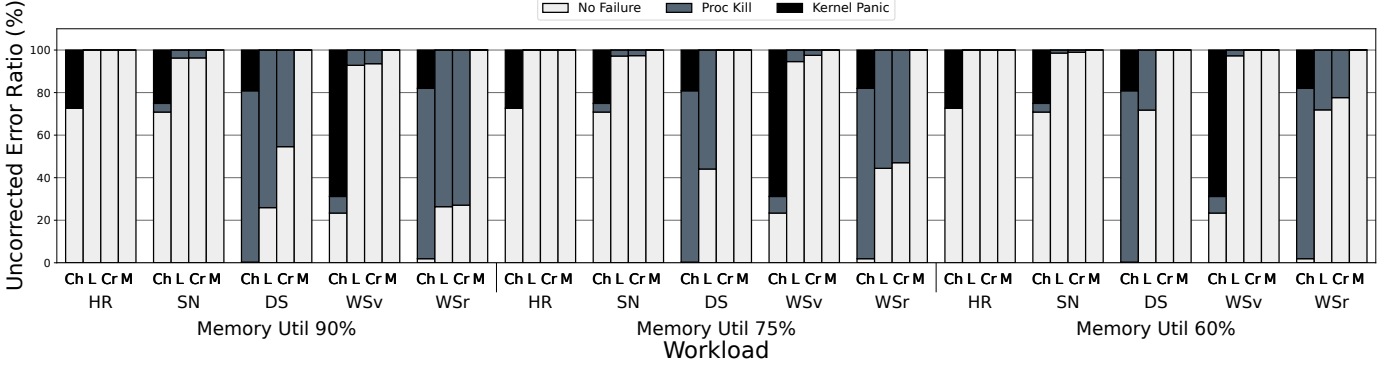


Fig. 11: Uncorrected Error Type Ratio of Chipkill-only (Ch), Lenovo (L), MemSOS (Criticality-only) (Cr), and MemSOS (M).

accesses (e.g., mirrored writes, metadata lookups) to estimate their impact on system throughput. To reflect real deployment conditions, we enable patrol scrubbing with a 24-hour interval—a common reliability feature in production systems [1], [10], [42], [66], [71]. Throughput is measured as requests/sec for DeathStarBench and operations/sec for CloudSuite.

For software-centric operations (mirror candidate selection and removal via LRU updates in Mirror Selection Daemon), we use the `bcc` eBPF toolkit to measure performance on the real system. We record both end-to-end wall-clock time and per-function execution time (e.g., mirror selection, LRU updates), enabling fine-grained attribution of overhead.

VII. EVALUATION RESULTS

A. Reliability Improvement

Figure 10 presents the FIT of MemSOS and the comparative baselines, normalized to a Chipkill-only system. MemSOS achieves up to 19,000 $\times$ lower FIT than Lenovo, while closely approaching the reliability of full mirroring. While reliability typically degrades at higher memory utilization due to limited mirroring space, MemSOS still achieves an average 12,000 $\times$ improvement over Lenovo at 90% memory utilization, with FIT only about 5 $\times$ higher than full mirroring. For workloads with a high proportion of file pages, such as Data Serving (DS), considering only criticality (i.e., MemSOS (Criticality-only)) already provides substantial reliability gains.

Although absolute reliability improves at 60% utilization due to more available space for mirroring, the relative benefit

of MemSOS over Criticality-only mirroring becomes dramatically more pronounced at 90%. Specifically, the reliability improvement between the two approaches increases by 80 $\times$ compared to 60% utilization. This amplification occurs because, under high memory pressure, mirroring all critical pages—particularly anonymous ones—becomes infeasible without incorporating recency into the selection process.

Figure 11 shows the proportion of uncorrectable errors detected through reads. *No Failure* indicates cases where uncorrectable errors are successfully recovered—either through mirroring or by reading clean file-backed pages from disk. *Process Kill* occurs when errors are encountered in anonymous or dirty file-backed pages, leading to process termination, while *Kernel Panic* represents read failures in kernel-space pages that result in system crashes. Workloads dominated by anonymous page accesses, such as Data Serving and Web Search, tend to exhibit higher rates of *Process Kill*. In contrast, workloads with frequent kernel-space reads (e.g., Web Serving) show elevated *Kernel Panic* rates, underscoring the importance of kernel-focused strategies (e.g., Lenovo).

B. Performance Overhead

MemSOS incurs runtime overhead from four operations: mirror candidate selection, mirror creation, mirrored write handling, and mirror removal. These are categorized as (1) *periodic* tasks—triggered every sampling interval—and (2) *on-demand* tasks—caused by memory writes or reclaims, including those from the kernel. Below, we quantify each.

Mirror Candidate Selection (Periodic). Every 200ms, Mirror Selection Daemon samples access patterns via PMU, updates LRU lists, and mirrors the top- k non-mirrored pages in $O(k)$ time. If mirror space is full, it evicts lower-priority mirrors to insert higher-priority ones based on criticality and recency. Selection and replacement complete within $20\mu s$ ($< 0.01\%$ of the interval), and PMU sampling adds no more than 1% overhead, keeping the total overhead under 1%.

Mirror Creation (Periodic). Every 200 ms, Mirror Selection Daemon mirrors newly selected pages by issuing one read and one write per page (plus metadata). A throttle limits this to 2000 mirrors/s, keeping overhead below 0.1% across all workloads.

Mirrored Write Handling (On-Demand). Integrating Mirror Manager into the memory controller introduces duplicate writes and occasional metadata lookups (*e.g.*, MMLB misses). Compared to the Chipkill-only baseline, the worst-case throughput drop was 1.53% in Social Network, where mirrored writes are frequent. Both Lenovo’s random user-page mirroring and MemSOS (Criticality-only) exhibit similar degradation in this workload, which contains write-intensive regions. However, MemSOS (Criticality-only) incurs lower overhead because it excludes file pages from mirroring, thereby avoiding extra writes to those pages. We observed no measurable channel contention; any potential effects likely remained below our detection threshold, thanks to hardware interleaving and selective mirroring.

Mirror Removal (On-Demand). When memory pressure increases, Mirror Selection Daemon scans its LRU lists to evict k mirrored pages. The operation has $O(k)$ complexity: it simply clears the corresponding bits in Mirror Bitmap. Since the Mirror Bitmap Cache exhibits a high hit ratio, the cost of invalidation is negligible. The more heavyweight Mirror Mapping Table entries remain intact after mirror removal and are lazily updated upon reassigning a mirror to the same page. We measured mirror-removal cost by triggering removals during allocation requests of different sizes (1–128 GB), where physical pages were instantiated via minor page faults. Our mirror-removal logic added 19.7% to allocation latency (*e.g.*, 6.7s for 128GB) with near-linear scaling. While minor page faults cause about 2% runtime overhead in cloud and VM environments—consistent with our measurements and prior work [6]—our logic applies only when new physical pages are allocated, yielding an overall performance impact of 0.33%.

Worst-Case Aggregate Overhead. Although these operations rarely incur their peak overheads simultaneously, even the worst-case concurrent scenario keeps total overhead below 3%: 1.53% from mirrored write handling (Fig. 12), 1% from candidate selection, 0.33% from removal, and 0.1% from creation.

Error Recovery Overhead. We evaluate steady-state performance without DRAM errors; rare recovery events are modeled separately. During recovery, the controller performs ECC decoding in the read path for on-the-fly correction. In a DDR5 $\times 4$ RDIMM with 8 data and 2 parity chips per-channel, a Chipkill-capable ECC corrects single-chip failures

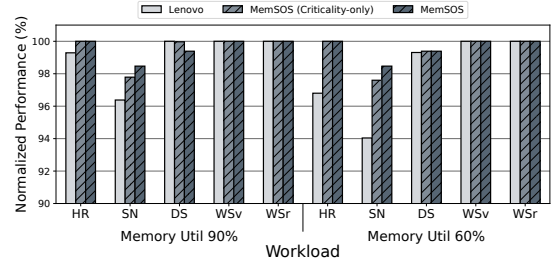


Fig. 12: Performance Overhead of Mirrored Write Handling

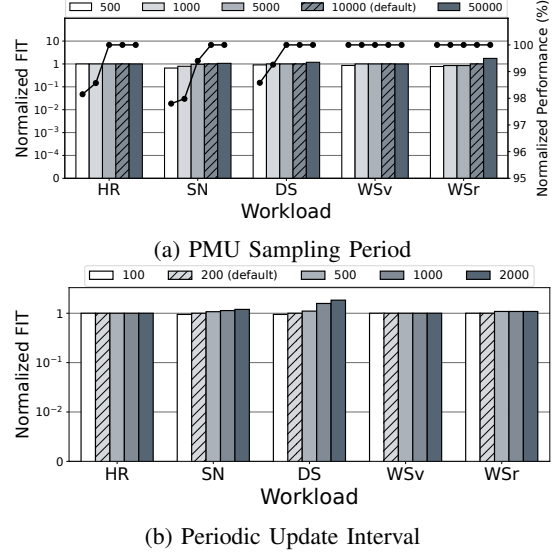


Fig. 13: Reliability Sensitivity to the Design Parameters

within one normal read since all 10 chips are accessed in parallel; the added latency from ECC decoding is only a few nanoseconds [50]. With mirroring, the controller accesses the mirrored page only if the original page still reports an uncorrectable error after Chipkill correction, which makes the worst-case recovery latency up to $4\times$ that of a normal read

C. Sensitivity Study

In this section, we conduct sensitivity study by varying five design parameters that may affect reliability and/or performance: 1) sampling period, 2) mirror creation rate, 3) cache size, 4) periodic update interval, and 5) mirroring granularity. **Sampling Period.** Longer sampling periods reduce PMU overhead but decrease recency accuracy. Figure 13(a) shows FIT normalized to the default period (10,000), across periods of 500–50,000. Workloads with dynamic access patterns (*i.e.*, Social Network, Data Serving, Web Search) observe up to 25% reliability improvement when decreasing the period to 1000. In contrast, stable workloads (*i.e.*, Hotel Reservation, Web Serving) show under 1% improvement. Performance degradation remains below 1% for most cases, except for read-intensive workloads (*i.e.*, Hotel Reservation, Social Network, Data Serving), which show up to 2.2% drops at sampling periods below 1000.

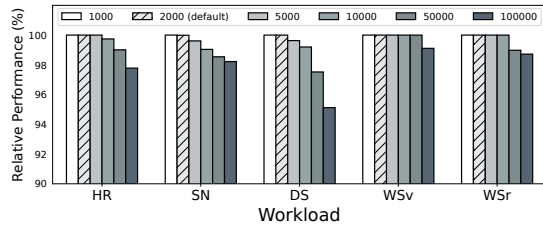


Fig. 14: Performance Sensitivity to the Mirror Creation Rate

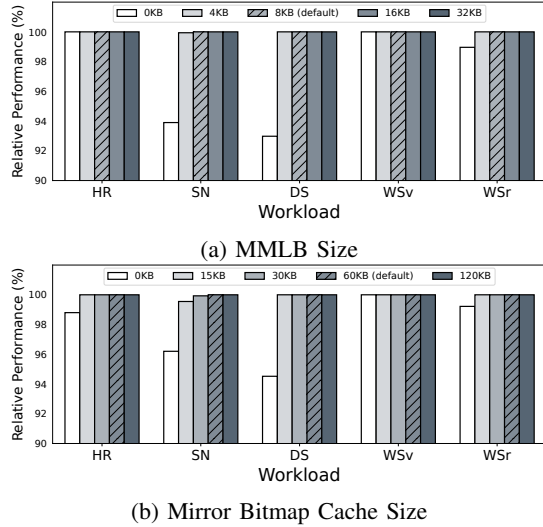


Fig. 15: Performance Sensitivity to the Cache Size

Mirror Creation Rate. As the mirror creation rate increases, reliability improves but may delay foreground requests, impacting performance. We evaluate this overhead using the same methodology as for duplicate writes, sweeping rates from 1,000 to 100,000 mirrors per second. Figure 14 shows the relative performance degradation of each workload compared to the baseline. Most workloads show under 0.1% performance degradation at up to 2000 mirrors/sec, with negligible reliability differences across rates. To balance reliability and performance, MemSOS defaults to 2000 mirrors/sec.

Cache Size. Larger caches generally reduce miss ratios and improve performance but increase power and area costs. Our design uses two caches: Mirror Bitmap Cache and MMLB. Mirror Bitmap Cache requires a larger size due to bitmap checks on all writes, while MMLB, accessed only for mirrored pages, performs well with a smaller size. We evaluate MMLB (0–16 KB) and Mirror Bitmap Cache (0–120 KB), with 0 KB disabling the cache. As shown in Figures 15(a) and 15(b), both caches substantially reduce metadata access latency. Disabling either cache causes up to 7% performance loss, while removing both increases it to 9.7% and 10.3% for the Social Network and Data Serving workloads, respectively.

Periodic Update Interval. Shorter update intervals improve recency accuracy but increase noise sensitivity. Figure 13(b) shows FIT normalized to the default interval (200ms), across 100–2000ms. We use 200ms by default, as more frequent

updates offer little reliability gain while maintaining a moderate mirror creation rate. As with sampling period sensitivity, dynamic workloads show up to 5% FIT improvement when reducing the interval from 200ms to 100ms, while stable workloads see <0.1% improvement across all intervals. Performance impact remains negligible regardless of the interval.

Mirroring Granularity. Larger mirroring units reduce metadata overhead but diminish recency tracking precision. We evaluate mirroring granularity from 4KB to 2 MB. Reliability remains stable at 4–32KB, drops at 64KB, and degrades significantly at 2MB—up to 340× higher FIT—because they include pages of varying reliability importance, reducing selection accuracy. MemSOS mirrors data at the compound-page level—the kernel’s abstraction for physically contiguous multi-page allocations (e.g., 2MB huge pages). Compound pages are tracked in the LRU list via their head page only, enabling coarser-grained management while reducing LRU overhead. While this simplifies metadata handling, it may reduce efficiency when critical data are sparse. Future work could explore adaptive granularity for selective mirroring. For instance, Memtis [41] tracks access locality to retain or split huge pages, a mechanism that could help mirror only frequently accessed subpages within compound pages.

D. Impact of MemSOS on Paging Performance

The mirror candidate selection task proactively updates the existing OS LRU list. This may affect the paging performance (i.e., the number of page faults) under extreme memory pressure, where swap and flush consistently occur. To investigate this possibility, we measure the total number of major page faults when running workloads for a fixed period after page swap is observed, for both the system with MemSOS and the baseline system. Even under extreme memory pressure, the impact on paging performance remains negligible. While the page fault count varies over workloads, the geomean of the page fault ratios is 0.97, that is, the system with MemSOS incurs 3% fewer page faults than the baseline system.

E. Compatibility with Modern Kernels

To demonstrate portability, we ported MemSOS to Linux v6.9.0, which adopts folio-based memory and Multi-Generational LRU (MGLRU). A folio is a more efficient abstraction for managing groups of contiguous pages. It generalizes compound pages (e.g., huge pages), enabling LRU list operations to treat multiple 4KB pages as a single entity. While this coarse granularity enables bulk operations, it may reduce accuracy in tracking recency and criticality. MGLRU replaces the active/inactive LRU with multi-level generations sorted by access recency, potentially affecting MemSOS’s mirror selection.

With both MGLRU and folio support enabled, MemSOS maintained stable performance and reliability across all workloads. While folios can span large regions (e.g., 2MB), potentially leading to inefficient mirroring of cold subpages, our measurements show that such large folios were rarely active, even when Transparent Huge Pages were enabled.

TABLE VI: Performance Counter Results

	L2 Miss	LLC Miss	IPC	Memory BW	Perf.
HR	2%	2.2%	-3.3%	5.5%	-0.1%
SN	4.3%	10.3%	-3.1%	12.9%	-2.3%
DS	8.3%	7.3%	-1.3%	20.6%	-0.9%
WSv	2.3%	3.6%	-0.3%	2.4%	0%
WSr	2.8%	3.7%	-2.4%	1.9%	0%

This finding is consistent with prior studies on datacenter memory usage [8], [37], [48]. As a result, most mirrored units remained at 4KB granularity, preserving MemSOS’s fine-grained accuracy and limiting reliability loss. We also observed that folio-based management did not increase the overhead of LRU list updates or metadata operations, as folios are managed similarly to compound pages in the legacy kernel.

We measured the runtime overhead from MGLRU-specific instrumentation in MemSOS. Each invocation of the candidate-selection logic costs $\sim 20\mu\text{s}$ per 200ms window ($<0.01\%$), similar to the baseline LRU implementation, and page-fault rates remained stable. We also evaluated hardware-level effects using performance counters under the default setup in Table III with 60% memory utilization, representative of steady-state datacenter workloads [7], [32], [49]. As summarized in Table VI, relative to the Chipkill-only baseline, L2/LLC misses increased by up to 10.3%, IPC decreased by under 3.3%, and memory bandwidth increased by at most 20.6%, resulting in an overall performance degradation of up to 2.3%. These results show that MemSOS incurs minimal hardware overhead under folio-based MGLRU.

F. Area and Power Estimation

We estimate the area and power overhead of Mirror Manager in the memory controller. As modern systems already support mirroring (e.g., Intel Address Range Mirror [21], [24], [29]), we exclude baseline mirroring logic and consider only the new SRAM structures, MMLB and Mirror Bitmap Cache. Using CACTI-P [47], we evaluate their overhead at 40nm and scale to 7nm following Biswas and Chandrakasan [3]. The normalized area costs are 0.0038mm^2 for MMLB and 0.019mm^2 for Mirror Bitmap Cache, proportional to their sizes. Power differs more noticeably— 0.619mW and 24.13mW , respectively—due to Mirror Bitmap Cache’s higher access frequency. Both structures are instantiated per core. Prior work estimates a 7nm DDR5 controller at roughly $3.5\text{--}4.0\text{mm}^2$ per channel [5], [56], and reports 10.6W power for a 16-core I/O die [68]. Against these baselines, MMLB and Mirror Bitmap Cache add under 1% area and about 3.7% power, indicating that Mirror Manager’s hardware cost is minimal.

VIII. RELATED WORK

Leveraging Unused Memory. Prior studies [7], [32], [40], [46], [49], [62], [69] have shown low memory utilization in real systems. Approaches such as Dvé [61] replicate frequently accessed pages in local memory to improve performance by reducing remote memory latency and enhancing reliability.

Similarly, Panwar et al. [57] use underutilized ranks to replicate data and bypass refresh delays, to enhance performance. While our approach also exploits unused memory, it targets reliability under memory constraints. Unlike Dvé, which does not account for memory pressure, our method selectively mirrors critical and recently accessed regions, implemented through an OS–HW co-design for efficient and reliable integration.

Hardware RAS Features for Memory. Hardware RAS features address DRAM faults by managing repeated error patterns. For example, Post Package Repair (PPR) detects and repairs defective rows at boot time by remapping them to spare rows [27]. Adaptive Double DRAM Device Correction (ADDDC) dynamically identifies devices experiencing frequent correctable errors (CEs), proactively remapping them to prevent potential uncorrectable errors (UEs) [22]. However, their focus on predictable errors limits effectiveness against random faults. Our approach complements these mechanisms by selectively mirroring high-risk pages based on criticality and access patterns, broadening fault coverage and enhancing defense against both predictable and unpredictable errors.

Memory RAS Feature with OS. Operating systems support memory RAS by using hardware error reports (e.g., Machine Check Exception) to mark faulty memory offline. Early work proposed the X/T rule for CEs, offlining pages with X errors in T hours [11], [20], [67]. Offlined pages remain unavailable until reboot or permanent retirement [52], so misprediction wastes memory. To improve accuracy, later studies analyzed error histories [15], [16] and introduced indicators such as risky CEs [45], all using CE trends to prevent future UEs. Mirroring enables lazy offlining and broadens analyzable error types by providing data on both CEs and uncorrectable-but-recoverable ones.

IX. CONCLUSION

Memory reliability is a critical issue in datacenters. While memory mirroring complements ECC for higher reliability, traditional approaches struggle to balance this with memory capacity. MemSOS tackles this through page-level partial mirroring, leveraging criticality and recency to optimize free memory use. By integrating mirror selection into the OS and management into the memory controller, MemSOS delivers up to $19,000\times$ improved reliability over existing methods with minimal performance overhead. This makes MemSOS ideal for modern datacenters that demand both reliability and efficiency under fluctuating system loads.

ACKNOWLEDGMENTS

This work was supported by an Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea Government (MSIT) (RS-2024-00456287), and in part by another IITP grant funded by the Korea Government (MSIT) (No. RS-2024-00459774, RISC-V based system software development for open ecosystem of SDR). Jae W. Lee is the corresponding author.

REFERENCES

- [1] M. V. Beigi, Y. Cao, S. Gurumurthi, C. Recchia, A. Walton, and V. Sridharan, "A Systematic Study of DDR4 DRAM Faults in the Field," in *Proceedings of the 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 991–1002.
- [2] M. V. Beigi, Y. Cao, S. Gurumurthi, C. Recchia, A. Walton, and V. Sridharan, "A systematic study of ddr4 dram faults in the field," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 991–1002.
- [3] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An Energy-Efficient SRAM With In-Memory Dot-Product Computation for Low-Power Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, 2019.
- [4] I. Boixaderas, D. Zivanovic, S. Moré, J. Bartolome, D. Vicente, M. Casas, P. M. Carpenter, P. Radojković, and E. Ayguadé, "Cost-Aware Prediction of Uncorrected DRAM Errors in the Field," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020, pp. 1–15.
- [5] M. N. Bojnordi and E. Ipek, "Pardis: A programmable memory controller for the ddrx interfacing standards," in *2012 39th Annual International Symposium on Computer Architecture (ISCA)*, 2012, pp. 13–24.
- [6] B. Caldwell, S. Goodarzi, S. Ha, R. Han, E. Keller, E. Rozner, and Y. Im, "Fluidmem: Full, flexible, and fast memory disaggregation for the cloud," in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, 2020, pp. 665–677.
- [7] W. Chen, K. Ye, Y. Wang, G. Xu, and C.-Z. Xu, "How Does the Workload Look Like in Production Cloud? Analysis and Clustering of Workloads on Alibaba Cluster Trace," *IEEE 24th International Conference on Parallel and Distributed Systems (ICPADS)*, pp. 102–109, 2018.
- [8] G. Choi, J. Son, J. Choi, S.-j. Cho, and Y. Won, "HPanal: a framework for analyzing tradeoffs of huge pages," in *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC '19, 2019, p. 1438–1443.
- [9] K. C. Chun, Y. K. Kim, Y. Ryu, J. Park, C. S. Oh, Y. Y. Byun, S. Y. Kim, D. H. Shin, J. G. Lee, B.-K. Ho, M.-S. Park, S.-J. Cho, S. Woo, B. M. Moon, B. Kil, S. Ahn, J. H. Lee, S. Y. Kim, S.-K. Choi, J.-S. Jeong, S.-G. Ahn, J. Kim, J. J. Kong, K. Sohn, N. S. Kim, and J.-B. Lee, "A 16-GB 640-GB/s HBM2E DRAM With a Data-Bus Window Extension Technique and a Synergetic On-Die ECC Scheme," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 1, pp. 199–211, 2021.
- [10] Cisco, "Cisco UCS Server BIOS Tokens in Intersight Managed Mode," 2021. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/unified_computing/ucs/Intersight/IMM_BIOS_Tokens_Guide/b_IMM_Server_BIOS_Tokens_Guide/m-processor.html
- [11] C. H. A. Costa, Y. Park, B. S. Rosenberg, C.-Y. Cher, and K. D. Ryu, "A System Software Approach to Proactive Memory-Error Avoidance," in *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2014, pp. 707–718.
- [12] Dell, "Dell PowerEdge R630 Owner's Manual," 2023. [Online]. Available: https://www.dell.com/support/manuals/en-us/poweredge-r630/r630_om_pub/memory-mirroring?guid=guid-c028bc5d-cf38-4517-a533-bd1b8ba71ddc&lang=en-us&lwpr=rt
- [13] T. J. Dell, "A White Paper on the Benefits of Chipkill-Correct ECC for PC Server Main Memory," *IBM Microelectronics Division*, vol. 11, no. 1–23, pp. 5–7, 1997.
- [14] P. Drongowski, L. Yu, F. Swehosky, S. Suthikulpanit, and R. Richter, "Incorporating Instruction-Based Sampling into AMD CodeAnalyst," in *Proceedings of the 2010 IEEE International Symposium on Performance Analysis of Systems & Software (ISPASS)*, 2010, pp. 119–120.
- [15] X. Du and C. Li, "Combining Error Statistics with Failure Prediction in Memory Page Offlining," in *Proceedings of the International Symposium on Memory Systems (MEMSYS)*, 2019, pp. 127–132.
- [16] X. Du, C. Li, S. Zhou, X. Liu, X. Xu, T. Wang, and S. Ge, "Fault-Aware Prediction-Guided Page Offlining for Uncorrectable Memory Error Prevention," in *Proceedings of the 2021 IEEE 39th International Conference on Computer Design (ICCD)*, 2021, pp. 456–463.
- [17] Fujitsu, "Memory Mirror and Memory Patrol for Improved System Reliability," 2023. [Online]. Available: <https://www.fujitsu.com/global/products/computing/servers/unix/sparc-enterprise/technology/availability/memory.html>
- [18] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rath, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvisky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou, "An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems," in *Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3–18.
- [19] S.-L. Gong, J. Kim, and M. Erez, "DRAM Scaling Error Evaluation Model Using Various Retention Time," in *Proceedings of the 2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, 2017, pp. 177–183.
- [20] A. A. Hwang, I. A. Stefanovici, and B. Schroeder, "Cosmic Rays Don't Strike Twice: Understanding the Nature of DRAM Errors and the Implications for System Design," *ACM SIGPLAN Notices*, vol. 47, no. 4, pp. 111–122, 2012.
- [21] Intel, "Address Range Partial Memory Mirroring," 2016. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/address-range-partial-memory-mirroring.html>
- [22] Intel, "New Reliability, Availability, and Serviceability (RAS) Features in the Intel® Xeon® Processor Family," 2017. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/new-reliability-availability-and-serviceability-ras-features-in-the-intel-xeon-processor.html>
- [23] Intel, "Intel Server Board M50CYP2SB Technical Product Specification," Intel Corporation, Tech. Rep., 2022, accessed: 2025-08-01. [Online]. Available: https://cdrdv2-public.intel.com/631280/M50CYP2SB_Server_Board_TPS_Rev1_41.pdf
- [24] Intel, "Intel Server D40AMP Family," 2022. [Online]. Available: <https://www.intel.com/content/dam/support/us/en/documents/server-products/multi-node-servers/d40amp-config-guide.pdf>
- [25] Intel, "Intel Server System M50CYP1UR Technical Product Specification," Intel Corporation, Tech. Rep., 2022, accessed: 2025-08-01. [Online]. Available: https://cdrdv2-public.intel.com/617333/M50CYP1UR_1U_TPS_1_42.pdf
- [26] Intel, "Intel® Xeon® Processor Scalable Family Technical Overview," 2022. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/xeon-processor-scalable-family-technical-overview.html>
- [27] Intel, "Post Package Repair (PPR)," 2023. [Online]. Available: <https://edc.intel.com/content/www/kr/ko/design/ipla/software-development-platforms/client/platforms/alder-lake-desktop/12th-generation-intel-core-processors-datasheet-volume-1-of-2/002/006/post-package-repair-ppr/>
- [28] Intel, "Timed Process Event Based Sampling," 2023. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/timed-process-event-based-sampling-tpebs.html>
- [29] Intel, "4th Gen Intel Xeon Scalable Processors: Reliability, Availability, and Serviceability," 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/775813/4th-gen-intel-xeon-scalable-processors-reliability-availability-and-serviceability-technical-paper.html?wapkw=775813>
- [30] Intel, "4th Gen Intel® Xeon® Processor Scalable Family, Codename Sapphire Rapids," 2024. [Online]. Available: https://cdrdv2-public.intel.com/814093/814093_SPR_Datasheet_Vol1_Rev1p0.pdf?utm_source=chatgpt.com
- [31] ITIC, "ITIC Reports and Survey Results," 2024. [Online]. Available: <https://itic-corp.com/itic-reports-surveys/>
- [32] W. Jin, W. Jang, H. Park, J. Lee, S. Kim, and J. W. Lee, "DRAM Translation Layer: Software-Transparent DRAM Power Savings for Disaggregated Memory," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–13.
- [33] J. Jung and M. Erez, "Predicting Future-System Reliability with a Component-Level DRAM Fault Model," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 944–956.
- [34] D. Kim, J. Lee, W. Jung, M. Sullivan, and J. Kim, "Unity ECC: Unified Memory Protection Against Bit and Chip Errors," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.
- [35] J. Kim, M. Sullivan, and M. Erez, "Bamboo ECC: Strong, Safe, and Flexible Codes for Reliable Computer Memory," in *Proceedings of the 2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, 2015, pp. 101–112.

- [36] J. Kim, M. Sullivan, S. Lym, and M. Erez, "All-Inclusive ECC: Thorough End-to-End Protection for Reliable Computer Memory," in *Proceedings of the 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 622–633.
- [37] Y. Kwon, H. Yu, S. Peter, C. J. Rossbach, and E. Witchel, "Coordinated and Efficient Huge Page Management with Ingens," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, ser. OSDI '16, 2016, pp. 705–721.
- [38] J. Lee, M. J. Kim, W.-S. Kim, and Y. S. Kim, "Review of Memory RAS for Data Centers," *IEEE Access*, vol. 11, pp. 124782–124796, 2023.
- [39] S. Lee, N.-H. Lee, K. W. Lee, J. H. Kim, J. H. Jin, Y. S. Lee, Y. C. Hwang, H. S. Kim, and S. Pae, "Development and Product Reliability Characterization of Advanced High Speed 14nm DDR5 DRAM with On-Die ECC," in *Proceedings of the 2023 IEEE International Reliability Physics Symposium (IRPS)*, 2023, pp. 1–4.
- [40] S. Lee, K.-D. Kang, H. Lee, H. Park, Y. Son, N. S. Kim, and D. Kim, "GreenDIMM: OS-Assisted DRAM Power Management for DRAM with a Sub-Array Granularity Power-Down State," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 131–142.
- [41] T. Lee, S. K. Monga, C. Min, and Y. I. Eom, "MEMTIS: Efficient Memory Tiering with Dynamic Page Classification and Page Size Determination," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 17–34.
- [42] Lenovo, "Lenovo's Best Practices in Response to the Intel® Uncorrectable Memory Error Handling on Gen 1, Gen 2 or "H" SKUs of Gen 3 Xeon® Scalable Processors," 2021. [Online]. Available: <https://support.lenovo.com/kr/ko/solutions/ht512486-best-practices-in-reducing-thinksystem-crashes-due-to-uncorrectable-memory-error-lenovo-thinksystem>
- [43] Lenovo, "Memory Mirroring Mode," 2023. [Online]. Available: https://pubs.lenovo.com/st650-v3/memory_module_installation_order_mirroring
- [44] Lenovo, "Using Memory Mirroring and Address Range Mirroring in VMware ESXi on Lenovo ThinkSystem Servers," 2023. [Online]. Available: <https://lenovopress.lenovo.com/lp1877-using-memory-mirroring-and-address-range-mirroring-in-esxi>
- [45] C. Li, Y. Zhang, J. Wang, H. Chen, X. Liu, T. Huang, L. Peng, S. Zhou, L. Wang, and S. Ge, "From Correctable Memory Errors to Uncorrectable Memory Errors: What Error Bits Tell," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022, pp. 1–14.
- [46] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini, "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023, pp. 574–587.
- [47] S. Li, K. Chen, J. H. Ahn, J. B. Brockman, and N. P. Jouppi, "CACTI-P: Architecture-Level Modeling for SRAM-Based Structures with Advanced Leakage Reduction Techniques," in *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, 2011, pp. 694–701.
- [48] X. Li, L. Liu, S. Yang, L. Peng, and J. Qiu, "Thinking about A New Mechanism for Huge Page Management," in *Proceedings of the 10th ACM SIGOPS Asia-Pacific Workshop on Systems*, ser. APSys '19, 2019, p. 40–46.
- [49] C. Lu, K. Ye, G. Xu, C.-Z. Xu, and T. Bai, "Imbalance in the Cloud: An Analysis on Alibaba Cluster Trace," in *Proceedings of the 2017 IEEE International Conference on Big Data (BigData)*, 2017, pp. 2884–2892.
- [50] E. Manzhosov, A. Hastings, M. Pancholi, R. Piersma, M. T. I. Ziad, and S. Sethumadhavan, "Revisiting residue codes for modern memories," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 73–90.
- [51] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "Revisiting Memory Errors in Large-Scale Production Data Centers: Analysis and Modeling of New Trends from the Field," in *Proceedings of the 2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2015, pp. 415–426.
- [52] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "Revisiting Memory Errors in Large-Scale Production Data Centers: Analysis and Modeling of New Trends from the Field," in *Proceedings of the 2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2015, pp. 415–426.
- [53] L. Mukhanov, K. Tovletoglou, H. Vandierendonck, D. S. Nikolopoulos, and G. Karakostas, "Workload-Aware DRAM Error Prediction Using Machine Learning," in *Proceedings of the 2019 IEEE International Symposium on Workload Characterization (IISWC)*, 2019, pp. 106–118.
- [54] P. J. Nair, D.-H. Kim, and M. K. Qureshi, "ArchShield: Architectural Framework for Assisting DRAM Scaling by Tolerating High Error Rates," in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*, 2013, pp. 72–83.
- [55] P. J. Nair, V. Sridharan, and M. K. Qureshi, "XED: Exposing On-Die Error Detection Information for Strong Memory Reliability," in *Proceedings of the 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 341–353.
- [56] M. V. Natale, M. Jung, K. Kraft, F. Lauer, J. Feldmann, C. Sudarshan, C. Weis, S. Krumke, and N. Wehn, "Efficient generation of application specific memory controllers," in *Proceedings of the International Symposium on Memory Systems*, ser. MEMSYS '20, 2021, p. 233–247.
- [57] G. Panwar, D. Zhang, Y. Pang, M. Dahshan, N. DeBardeleben, B. Ravindran, and X. Jian, "Quantifying Memory Underutilization in HPC Systems and Using It to Improve Performance via Architecture Support," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 821–835.
- [58] Parallel Systems Architecture Lab, "CloudSuite," 2023. [Online]. Available: <https://github.com/parsa-epfl/cloudsuite>
- [59] M. Patel, J. S. Kim, T. Shahroodi, H. Hassan, and O. Mutlu, "Bit-Exact ECC Recovery (BEER): Determining DRAM On-Die ECC Functions by Exploiting DRAM Data Retention Characteristics," in *Proceedings of the 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 282–297.
- [60] N. Patel, A. Mamandipoor, M. Nouri, and M. Alian, "Smartdim: In-memory acceleration of upper layer protocols," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 312–329.
- [61] A. Patil, V. Nagarajan, R. Balasubramonian, and N. Oswald, "Dvė: Improving DRAM Reliability and Performance On-Demand via Coherent Replication," in *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 526–539.
- [62] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis," *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC)*, pp. 7:1–7:13, 2012.
- [63] S. Shiratake, "Scaling and Performance Challenges of Future DRAM," in *Proceedings of the 2020 IEEE International Memory Workshop (IMW)*, 2020, pp. 1–3.
- [64] T. Siddiqua, A. E. Papathanasiou, A. Biswas, and S. Gurumurthi, "Analysis and Modeling of Memory Errors from Large-Scale Field Data Collection," in *SELS*, 2003.
- [65] V. Sridharan and D. Liberty, "A Study of DRAM Failures in the Field," in *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2012, pp. 1–11.
- [66] SuperMicro, "SuperMicro User Manual," 2019. [Online]. Available: <https://www.supermicro.com/manuals/superserver/1U/MNL-2192.pdf>
- [67] D. Tang, P. Carruthers, Z. Totari, and M. W. Shapiro, "Assessment of the Effect of Memory Page Retirement on System RAS Against Hardware Faults," in *Proceedings of the International Conference on Dependable Systems and Networks (DSN)*, 2006, pp. 365–370.
- [68] TECHPOWERUP, "DDR5 Memory Performance Scaling with AMD Zen 5 — techpowerup.com," <https://www.techpowerup.com/review/ddr5-memory-performance-scaling-with-amd-zen-5/single-page.html>, 2024.
- [69] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes, "Borg: The Next Generation," in *Proceedings of the 15th European Conference on Computer Systems (EuroSys)*, 2020, pp. 1–14.
- [70] M. White, J. Qin, and J. B. Bernstein, "A Study of Scaling Effects on DRAM Reliability," in *Proceedings of the Annual Reliability and Maintainability Symposium*, 2011.
- [71] S. C. Woo, W. Elsasser, M. Hamburg, E. Linstadt, M. R. Miller, T. Song, and J. Tringali, "RAMPART: RowHammer Mitigation and Repair for Server Memory Systems," in *Proceedings of the International Symposium on Memory Systems (MEMSYS)*, 2024, pp. 1–15.
- [72] D. H. Yoon and M. Erez, "Virtualized and Flexible ECC for Main Memory," in *Proceedings of the 15th International Conference on Ar-*

chitectural Support for Programming Languages and Operating Systems (ASPLOS), 2010, pp. 397–408.

- [73] Q. Yu, W. Zhang, J. Cardoso, and O. Kao, “Exploring Error Bits for Memory Failure Prediction: An In-Depth Correlative Study,” in *Proceedings of the 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [74] J. Zhang, M. Swift, and J. J. Li, “Software-defined address mapping: a case on 3d memory,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’22. Association for Computing Machinery, 2022, p. 70–83.